



Stack tech nel web dev

Web dev

Nel Web Development, uno stack tecnologico è l'insieme degli strumenti, linguaggi, framework, librerie, database e servizi utilizzati per progettare, sviluppare, distribuire e mantenere un'applicazione web.

In altre parole, rappresenta la “combinazione tecnica” scelta per costruire un sito, una web app o una piattaforma digitale.

Uno stack web moderno è solitamente diviso in più livelli. Il primo è il front-end, cioè la parte visibile all'utente: interfaccia grafica, pagine, pulsanti, form, animazioni e interazioni.



Qui troviamo tecnologie come HTML, CSS, JavaScript e framework come React, Angular o Vue.

Il secondo livello è il back-end, cioè la parte logica dell'applicazione. Il back-end gestisce dati, autenticazione, autorizzazioni, comunicazione con il database, API, sicurezza e regole di business.

Alcuni linguaggi e framework comuni sono Node.js con Express, Python con Django o Flask, Java con Spring Boot, PHP con Laravel e C# con ASP.NET.

Un altro elemento fondamentale dello stack è il database, usato per salvare e recuperare informazioni. I database possono essere relazionali, come MySQL, PostgreSQL e SQL Server, oppure NoSQL, come MongoDB, Redis o Firebase Firestore.



La scelta dipende dal tipo di dati, dalla struttura del progetto e dalle esigenze di scalabilità.

Oltre a front-end, back-end e database, uno stack moderno include spesso anche strumenti di versionamento, deploy, cloud, containerizzazione e monitoraggio.

Per esempio, Git e GitHub vengono usati per gestire il codice, Docker per creare ambienti coerenti, servizi cloud come AWS, Azure o Google Cloud per pubblicare l'applicazione, e strumenti di CI/CD per automatizzare test e distribuzione.

La scelta dello stack tecnologico non è casuale: dipende dal tipo di progetto, dal team, dal budget, dalle prestazioni richieste, dalla scalabilità futura e dalle competenze disponibili.



Non esiste uno stack “migliore” in assoluto, ma esiste lo stack più adatto a uno specifico contesto.

MERN Stack - MongoDB, Express.js, React, Node.js

Il MERN Stack è uno degli stack JavaScript più diffusi per lo sviluppo di applicazioni web moderne.

Utilizza JavaScript o TypeScript sia nel front-end sia nel back-end, permettendo al team di lavorare con un linguaggio unico lungo quasi tutta l'applicazione.

React gestisce l'interfaccia utente, Express si occupa delle API lato server, Node.js esegue il back-end e MongoDB salva i dati in formato documentale.

È molto usato per dashboard, web app dinamiche, SaaS, gestionali e applicazioni single page.

React, Node.js ed Express risultano ancora tra le tecnologie web più rilevanti nei dati Stack Overflow 2025



MERN Stack - MongoDB, Express.js, React, Node.js

Casi d'uso principali:

- Applicazioni web dinamiche
- Dashboard interattive
- Gestionali moderni
- Single Page Application
- Piattaforme SaaS
- App con dati flessibili e non troppo rigidi
- Prototipi rapidi e MVP

Esempio: una piattaforma per gestire utenti, prodotti, statistiche e notifiche in tempo reale.



MERN Stack - MongoDB, Express.js, React, Node.js

Nel MERN Stack, uno dei problemi più comuni è la gestione della complessità lato front-end, soprattutto quando React cresce molto e servono regole chiare per componenti, stato globale, chiamate API e routing.

MongoDB, essendo molto flessibile, può diventare difficile da mantenere se non viene progettata bene la struttura dei documenti.

Inoltre, usando JavaScript sia lato client sia lato server, eventuali errori di organizzazione del codice possono propagarsi facilmente in tutta l'applicazione.



MEAN Stack - MongoDB, Express.js, Angular, Node.js

Il MEAN Stack è simile al MERN, ma al posto di React usa Angular. Anche qui il linguaggio principale è JavaScript/TypeScript, ma Angular offre una struttura più rigida e completa rispetto a React.

È indicato per applicazioni aziendali, progetti strutturati, gestionali complessi e piattaforme dove è importante avere regole architetturali chiare.

Angular include molte funzionalità già integrate, come routing, gestione dei form, servizi e dependency injection.

Rispetto a MERN, è spesso percepito come meno “leggero”, ma più ordinato nei progetti di grandi dimensioni è quindi adatto quando il team ha bisogno di una struttura solida e standardizzata.



MEAN Stack - MongoDB, Express.js, Angular, Node.js

Casi d'uso principali:

- Applicazioni enterprise
- Gestionali aziendali complessi
- Piattaforme amministrative
- CRM interni
- Applicazioni con molti moduli
- Progetti dove serve una struttura front-end molto organizzata
- Dashboard aziendali strutturate



Esempio: un gestionale interno per un'azienda con utenti, ruoli, report, permessi e moduli separati.

MEAN Stack - MongoDB, Express.js, Angular, Node.js

Nel MEAN Stack, Angular offre una struttura molto completa, ma può risultare più complesso da imparare rispetto ad altri framework front-end.

La curva di apprendimento è più alta perché bisogna comprendere componenti, moduli, servizi, dependency injection, RxJS e TypeScript.

Anche qui MongoDB può creare problemi se i dati non vengono modellati correttamente, soprattutto in applicazioni aziendali con molte relazioni tra entità.



LAMP Stack - Linux, Apache, MySQL, PHP

Il LAMP Stack è uno degli stack storici del web. È composto da Linux come sistema operativo, Apache come web server, MySQL come database relazionale e PHP come linguaggio back-end.

È stato alla base di moltissimi siti web, CMS, blog, e-commerce e applicazioni server-side. Tecnologie come WordPress, Drupal e molte piattaforme PHP-based si appoggiano spesso a questo tipo di architettura.

Anche se oggi esistono stack più moderni, LAMP resta importante perché è stabile, economico, ben documentato e supportato praticamente da qualsiasi hosting.

È ideale per siti web tradizionali, portali aziendali e progetti dove serve affidabilità più che sperimentazione.



LAMP Stack - Linux, Apache, MySQL, PHP

Casi d'uso principali:

- Siti web tradizionali
- Blog
- Portali aziendali
- CMS come WordPress
- E-commerce classici
- Aree riservate semplici
- Applicazioni web server-side

Esempio: un sito aziendale con pagine informative, blog, form di contatto e area amministrativa.



LAMP Stack - Linux, Apache, MySQL, PHP

Nel LAMP Stack, un problema frequente è la presenza di codice legacy, cioè applicazioni sviluppate anni prima con logiche poco moderne o difficili da mantenere.

PHP è molto flessibile, ma se non viene usato con buone pratiche può portare a codice disordinato e poco sicuro.

Inoltre, su progetti molto grandi, la separazione tra logica, interfaccia e database può diventare confusa se non si utilizza un framework o una buona architettura.



PERN Stack - PostgreSQL, Express.js, React, Node.js

Il PERN Stack è una variante del MERN in cui MongoDB viene sostituito da PostgreSQL, un database relazionale avanzato. Questo lo rende molto adatto quando i dati hanno relazioni forti, vincoli, transazioni e interrogazioni complesse.

React gestisce il front-end, Express e Node.js il back-end, mentre PostgreSQL si occupa della persistenza dei dati. È uno stack molto usato in applicazioni gestionali, piattaforme business, CRM, sistemi amministrativi e prodotti SaaS.

La sua forza è unire la modernità dell'ecosistema JavaScript con la solidità di un database SQL.



È spesso preferibile a MERN quando i dati non sono semplici documenti, ma entità fortemente collegate tra loro.

PERN Stack - PostgreSQL, Express.js, React, Node.js

Casi d'uso principali:

- Applicazioni con dati relazionali complessi
- CRM
- ERP leggeri
- Gestionali amministrativi
- Piattaforme SaaS business
- Sistemi con transazioni e vincoli sui dati
- Applicazioni dove servono query SQL avanzate

Esempio: una piattaforma per gestire clienti, ordini, fatture, pagamenti e report collegati tra loro.



PERN Stack - PostgreSQL, Express.js, React, Node.js

Nel PERN Stack, la difficoltà principale riguarda la corretta progettazione del database relazionale.

PostgreSQL è potente, ma richiede attenzione nella definizione di tabelle, relazioni, vincoli, indici e query.

Se la struttura dati non è progettata bene, l'applicazione può diventare lenta o difficile da modificare.

Inoltre, come nel MERN, React e Node.js richiedono una buona organizzazione del codice per evitare confusione tra front-end, API e logica applicativa.



Py Django - Python, Django, PostgreSQL/MySQL, React/Vue

Lo stack basato su Python e Django è molto usato per applicazioni web robuste, sicure e veloci da sviluppare. Django è un framework back-end “batteries included”, cioè include già molte funzionalità pronte: autenticazione, pannello admin, ORM, gestione URL, sicurezza e interazione con il database.

Può essere usato con template HTML tradizionali oppure come back-end API collegato a front-end moderni come React o Vue. È molto adatto per gestionali, piattaforme educative, applicazioni data-driven, portali interni, e-commerce e sistemi amministrativi.

Il vantaggio principale è la produttività: Django permette di costruire molto rapidamente applicazioni complete, mantenendo una buona organizzazione del codice e un buon livello di sicurezza.



Py Django - Python, Django, PostgreSQL/MySQL, React/Vue

Casi d'uso principali:

- Applicazioni gestionali
- Portali educativi
- Piattaforme data-driven
- Applicazioni con pannello admin
- E-commerce
- Sistemi interni aziendali
- Progetti dove serve sviluppo rapido e sicurezza integrata

Esempio: una piattaforma scolastica per gestire studenti, corsi, docenti, iscrizioni e valutazioni.



Py Django - Python, Django, PostgreSQL/MySQL, React/Vue

Nel Python Django Stack, un problema comune è affidarsi troppo alle funzionalità automatiche del framework senza comprenderne bene il funzionamento.

Django offre molti strumenti già pronti, ma questo può portare a progetti rigidi o difficili da personalizzare se non si conoscono bene ORM, middleware, autenticazione e struttura MVC/MVT.

Inoltre, per applicazioni altamente asincrone o real-time, Django tradizionale può richiedere componenti aggiuntivi e una progettazione più attenta.



JAMstack / Next.js Stack - JavaScript/TypeScript, Next.js, API, CMS headless, CDN

Il JAMstack è uno stack moderno basato sull'idea di separare il front-end dalla logica server tradizionale.

Le pagine possono essere generate in anticipo, servite tramite CDN e collegate a servizi esterni tramite API.

Una delle tecnologie più usate in questo contesto è Next.js, spesso associata a React, TypeScript, API serverless, CMS headless e piattaforme cloud come Vercel o Netlify.

Questo stack è molto usato per siti aziendali moderni, landing page, blog, e-commerce headless e applicazioni dove contano performance, SEO e velocità di distribuzione.



Tra i framework moderni, Next.js viene spesso citato tra le soluzioni più rilevanti per il web development contemporaneo.

JAMstack / Next.js Stack

Casi d'uso principali:

- Siti aziendali moderni
- Landing page
- Blog performanti
- E-commerce headless
- Siti con forte attenzione alla SEO
- Portali editoriali
- Applicazioni con contenuti serviti velocemente tramite CDN

Esempio: un sito marketing internazionale con pagine veloci, contenuti gestiti da CMS e ottimizzazione SEO.



JAMstack / Next.js Stack

Nel JAMstack o Next.js Stack, una delle difficoltà principali è capire bene quando usare pagine statiche, rendering lato server, API route o generazione dinamica.

Se queste scelte non sono chiare, il progetto può diventare complesso da gestire. Inoltre, l'integrazione con CMS headless, API esterne, autenticazione e funzioni serverless può creare dipendenze da molti servizi diversi.

Questo rende lo stack molto performante, ma anche più frammentato rispetto a un'applicazione monolitica tradizionale.



Buon Davante a tutti

